Deep Learning Practical No.2

Name - Vedant Joshi

Roll No.24

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
```

```
from tensorflow.keras.datasets import imdb

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)

print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000
Testing samples: 25000
```

```
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))

    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.0

    return results

x_train = vectorize_sequences(x_train)
x_test = vectorize_sequences(x_test)

y_train = np.asarray(y_train).astype("float32")
y_test = np.asarray(y_test).astype("float32")
```

```
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(10000,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')  # Binary output
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=512,
    validation_split=0.2
)
```

```
Epoch 1/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 4s 59ms/step - accuracy: 0.8055 - loss: 0.4928 - val_accuracy: 0.8802 - val_loss: 0.3289
Epoch 2/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.9119 - loss: 0.2485 - val_accuracy: 0.8910 - val_loss: 0.2766
Epoch 3/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - accuracy: 0.9401 - loss: 0.1786 - val_accuracy: 0.8916 - val_loss: 0.2774
Epoch 4/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9554 - loss: 0.1382 - val_accuracy: 0.8878 - val_loss: 0.2933
Epoch 5/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - accuracy: 0.9660 - loss: 0.1102 - val_accuracy: 0.8860 - val_loss: 0.3193
Epoch 6/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step - accuracy: 0.9728 - loss: 0.0906 - val_accuracy: 0.8824 - val_loss: 0.3550
Epoch 7/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.9808 - loss: 0.0739 - val_accuracy: 0.8774 - val_loss: 0.3856
Epoch 8/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9857 - loss: 0.0596 - val_accuracy: 0.8774 - val_loss: 0.4164
Epoch 9/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 2s 40ms/step - accuracy: 0.9914 - loss: 0.0465 - val_accuracy: 0.8768 - val_loss: 0.4504
Epoch 10/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.9936 - loss: 0.0366 - val_accuracy: 0.8730 - val_loss: 0.4954
```

```
results = model.evaluate(x_test, y_test)
```

```
print("Test Accuracy:", results[1])
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8596 - loss: 0.5335
Test Accuracy: 0.8596400022506714
```

```
predictions = model.predict(x_test)

print(predictions[:5])
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step
[[0.09389819]
 [0.9999894 ]
 [0.00996649]
 [0.7153758 ]
 [0.9932328 ]]
```

```
import matplotlib.pyplot as plt

history_dict = history.history

plt.plot(history_dict['accuracy'], label='Training Accuracy')
plt.plot(history_dict['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.show()
```